

You are required to deploy a Flask backend and an Express frontend application using **AWS** and **Terraform**. Follow the steps below to deploy the applications in different configurations.

Part 1: Deploy Both Flask and Express on a Single EC2 Instance

1. **Objective:** Deploy both the Flask backend and the Express frontend on a **single EC2 instance** using Terraform.
 - Provision the EC2 instance with Terraform.
 - Configure the instance with:
 - A user data script or a configuration management tool (e.g., Ansible or Cloud-Init) to install dependencies (Python, Node.js).
 - Scripts to start the Flask backend and the Express frontend.
 - Ensure both applications are running on different ports (e.g., Flask on port 5000 and Express on port 3000).
2. **Expected Deliverables:**
 - Terraform configuration files (`main.tf`, `variables.tf`, etc.).
 - A working EC2 instance with Flask and Express running and accessible via the instance's public IP.

The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree on the left showing the project structure for 'FLASK EXPRESS-AWS'. It includes folders for 'backend', 'frontend', and 'terraform', along with various configuration files like 'main.tf', 'variables.tf', and 'outputs.tf'.
- EDITOR:** The main workspace displays the 'main.tf' file, which contains Terraform configuration for an AWS VPC and subnets. The code is as follows:

```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 4.66.0"
6     }
7   }
8 }
9 data "aws_vpc" "default" {
10   default = true
11 }
12
13 data "aws_subnets" "default" {
14   filter {
15     name = "vpc-id"
16     values = [data.aws_vpc.default.id]
17   }
18 }
19
```
- TERMINAL:** A terminal window at the bottom shows the output of the command 'pm2 logs flask-backend'. It displays a table of running processes and their logs.

id	name	mode	pid	status	cpu	memory
1	express-frontend	fork	0	online	0%	55.7mb
0	flask-backend	fork	0	online	0%	23.6mb

The terminal output shows the following logs for the flask-backend process:

```
ubuntu@ip-172-31-0-57:~$ pm2 logs flask-backend
[TAILING] Tailing last 15 lines for [flask-backend] process (change the value with --lines option)
/home/ubuntu/.pm2/logs/flask-backend-out.log last 15 lines:
0|flask-ba | [2026-01-23 05:52:44 +0000] [9277] [INFO] Booting worker with pid: 9277
0|flask-ba | [2026-01-23 05:52:44 +0000] [9278] [INFO] Booting worker with pid: 9278
0|flask-ba | [2026-01-23 05:52:44 +0000] [9279] [INFO] Booting worker with pid: 9279
0|flask-ba | [2026-01-23 05:52:44 +0000] [9280] [INFO] Booting worker with pid: 9280
```

```
PS C:\AWS\flask-express-aws\terraform> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_eip.flask_express_eip will be created
+ resource "aws_eip" "flask_express_eip" {
  + allocation_id      = (known after apply)
  + arn                = (known after apply)
  + association_id     = (known after apply)
  + carrier_ip         = (known after apply)
  + customer_owned_ip  = (known after apply)
  + domain             = "vpc"
  + id                 = (known after apply)
  + instance            = (known after apply)
  + ipam_pool_id       = (known after apply)
  + network_border_group = (known after apply)
  + network_interface  = (known after apply)
  + private_dns        = (known after apply)
```

```
PS C:\AWS\flask-express-aws\terraform> terraform apply -auto-approve
```

```
data.aws_vpc.default: Reading...
```

```
aws_security_group.flask_express_sg: Refreshing state... [id=sg-045ecb466eb86ca99]
```

```
data.aws_vpc.default: Read complete after 3s [id=vpc-08d45953a32631ab2]
```

```
data.aws_subnets.default: Reading...
```

```
data.aws_subnets.default: Read complete after 0s [id=ap-south-1]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# aws_eip.flask_express_eip will be created
+ resource "aws_eip" "flask_express_eip" {
  + allocation_id      = (known after apply)
  + arn                = (known after apply)
```

```
+ ssh_command = (known after apply)
aws_instance.flask_express_instance: Creating...
aws_instance.flask_express_instance: Still creating... [00m10s elapsed]
aws_instance.flask_express_instance: Creation complete after 18s [id=i-053983c362b0e8341]
aws_eip.flask_express_eip: Creation complete after 3s [id=eipalloc-0775ab46cb1b5293e]
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
backend_url = "http://3.111.128.54:5000"
frontend_url = "http://3.111.128.54:3000"
instance_id = "i-053983c362b0e8341"
instance_public_dns = ""
instance_public_ip = "3.111.128.54"
ssh_command = "ssh -i window.pem ubuntu@3.111.128.54"
```

successfully processed 1 files; Failed processing 0 files

```
PS C:\AWS\flask-express-aws\terraform> ssh -i window.pem ubuntu@3.111.128.54
```

Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

```
* Documentation:  https://help.ubuntu.com
* Management:     https://landscape.canonical.com
* Support:         https://ubuntu.com/pro
```

System information as of Fri Jan 23 06:01:14 UTC 2026

```
System load:  0.0           Temperature:    -273.1 C
Usage of /:    41.1% of 6.71GB Processes:         119
Memory usage:  40%          Users logged in:  0
Swap usage:    0%           IPv4 address for ens5: 172.31.0.57
```

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

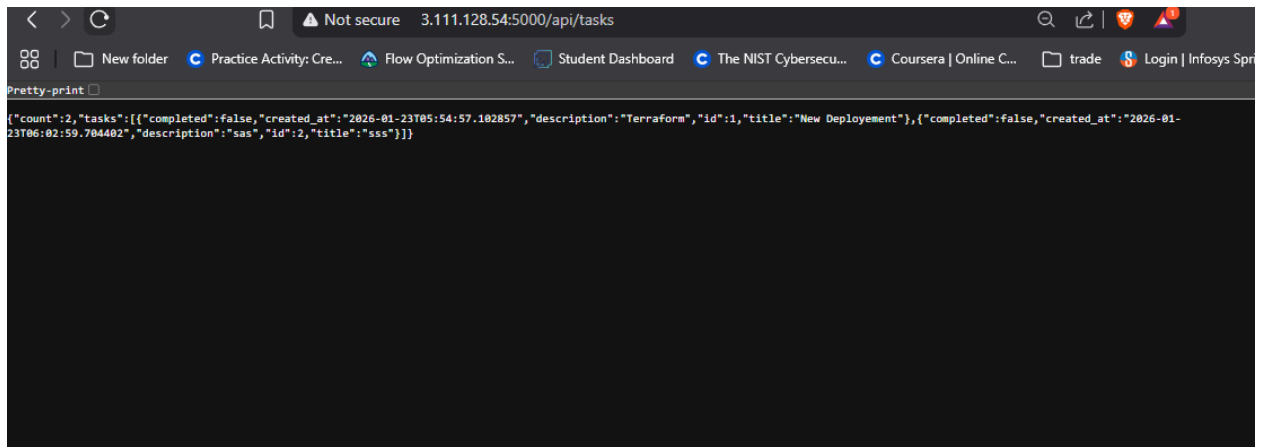
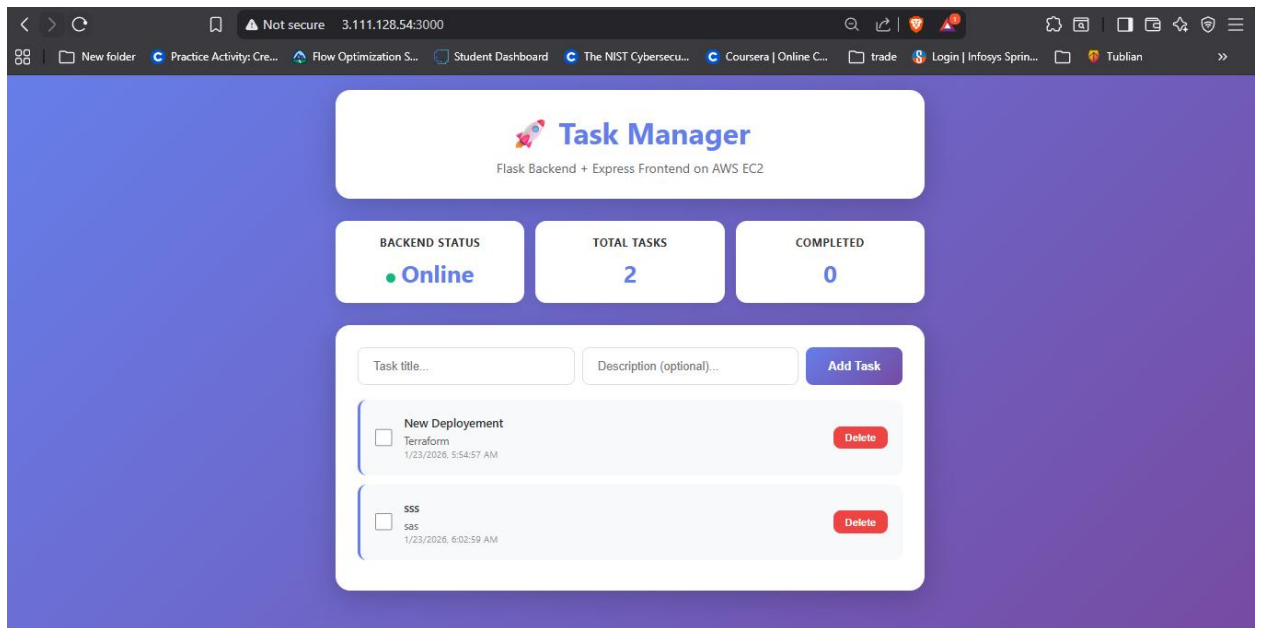
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

```
ubuntu@ip-172-31-0-57:~$ pm2 status
```

```
ubuntu@ip-172-31-0-57:~$ pm2 status
```

id	name	mode	id	status	cpu	memory
1	express-frontend	fork	0	online	0%	55.7mb
0	flask-backend	fork	0	online	0%	23.6mb



Part 2: Deploy Flask and Express on Separate EC2 Instances

1. **Objective:** Deploy the Flask backend and the Express frontend on **two separate EC2 instances** using Terraform.
 - Provision two EC2 instances using Terraform:
 - One for the Flask backend.
 - One for the Express frontend.
 - Configure security groups to:
 - Allow communication between the two instances.
 - Expose both applications to the internet on their respective ports.
2. **Configuration:**
 - Use Terraform to define networking resources, such as VPC, subnets, and security groups.
 - Use user data scripts to automate the installation and startup of both applications.
3. **Expected Deliverables:**
 - Terraform configuration files.
 - Two working EC2 instances: one running Flask and one running Express.
 - Security groups configured to allow proper communication and public access.

```
PS C:\AWS\project-root\terraform> terraform apply
data.aws_ami.ubuntu: Reading...
aws_vpc.main: Refreshing state... [id=vpc-060a43bdce3f8b022]
aws_internet_gateway.main: Refreshing state... [id=igw-09b7e1cd56a416c03]
aws_subnet.public[0]: Refreshing state... [id=subnet-0ffa5856009034f67]
aws_subnet.public[1]: Refreshing state... [id=subnet-0342969e7b67d2c8f]
aws_security_group.frontend: Refreshing state... [id=sg-05f718bf2eff031fe]
data.aws_ami.ubuntu: Read complete after 0s [id=ami-0ff91eb5c6fe7cc86]
aws_route_table.public: Refreshing state... [id=rtb-08095eb8c0f459d60]
aws_security_group.backend: Refreshing state... [id=sg-0608378b45fc8b57d]
aws_route_table_association.public[1]: Refreshing state... [id=rtbassoc-0814dac436be4a949]
aws_route_table_association.public[0]: Refreshing state... [id=rtbassoc-0d518ed943823fe8e]
```

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

```
# aws_internet_gateway.main has been deleted
- resource "aws_internet_gateway" "main" {
```

Apply complete! Resources: 13 added, 0 changed, 0 destroyed.

Outputs:

```
backend_instance_id = "i-0de422b70c2f4be50"
backend_private_ip = "10.0.1.5"
backend_public_ip = "3.111.73.198"
backend_url = "http://3.111.73.198:5000"
frontend_instance_id = "i-0ad2a677cb560f42a"
frontend_private_ip = "10.0.2.181"
frontend_public_ip = "13.232.235.190"
frontend_url = "http://13.232.235.190:3000"
ssh_commands = {
  "backend" = "ssh -i window.pem ubuntu@3.111.73.198"
  "frontend" = "ssh -i window.pem ubuntu@13.232.235.190"
}
vpc_id = "vpc-0852bcacf28c046007"
```

Not secure 13.232.235.190:3000

User Management System

Backend Connected

Add New User

Name:

Email:

Role:

Not secure 13.232.235.190:3000

Role:

Users List

Alice Johnson

alice@example.com

ID: 1

Bob Smith

bob@example.com

ID: 2

Carol White

carol@example.com

ID: 3

swapnil

swpnil234@gmail.com

ID: 4

```
Not secure 3.111.73.198:5000/api/users
Pretty-print
{
  "count": 5,
  "data": [
    {
      "email": "alice@example.com",
      "id": 1,
      "name": "Alice Johnson",
      "role": "Developer"
    },
    {
      "email": "bob@example.com",
      "id": 2,
      "name": "Bob Smith",
      "role": "Designer"
    },
    {
      "email": "carol@example.com",
      "id": 3,
      "name": "Carol White",
      "role": "Manager"
    },
    {
      "email": "pawartushar8485@gmail.com",
      "id": 4,
      "name": "Tushar",
      "role": "Developer"
    },
    {
      "email": "swp234@gmail.com",
      "id": 5,
      "name": "swapnil",
      "role": "User"
    }
  ],
  "success": true
}
```

Part 3: Deploy Flask and Express Using Docker and AWS Services

1. **Objective:** Deploy Flask and Express as **Docker containers** using **AWS ECR, ECS,** and **VPC** with Terraform.
2. **Steps:**
 - **ECR:**
 - Use Terraform to create two Elastic Container Registry (ECR) repositories: one for the Flask backend and one for the Express frontend.
 - Build Docker images for both applications and push them to their respective ECR repositories.
 - **VPC:**
 - Use Terraform to create a VPC with subnets, route tables, and security groups.
 - **ECS:**
 - Use Terraform to set up an ECS cluster.
 - Create two ECS services:
 - One for the Flask backend.
 - One for the Express frontend.
 - Use ECS Fargate or EC2 launch type to deploy the containers.
 - **Load Balancer:**
 - Use Terraform to provision an Application Load Balancer (ALB).
 - Configure ALB listeners to route requests to the appropriate ECS service.
3. **Expected Deliverables:**
 - Terraform configuration files for ECR, ECS, VPC, and ALB setup.
 - Docker images pushed to ECR.
 - ECS services running and accessible via the ALB.

General Requirements

- Ensure that all Terraform configurations follow best practices, including the use of `variables.tf` and `outputs.tf`.
- Use **Terraform State** (e.g., S3 backend) for state management.
- Use `terraform plan` and `terraform apply` to validate and deploy infrastructure.
- Document the deployment process in a `README.md` file.

```
PS C:\AWS\New folder\flask-express-ecs\scripts> ./deploy.sh
PS C:\AWS\New folder\flask-express-ecs\scripts>
```

```
aws_lb.backend: Still creating... [02m30s elapsed]
aws_lb.frontend: Still creating... [02m30s elapsed]
aws_lb.backend: Creation complete after 2m32s [id=arn:aws:elasticloadbalancing:ap-south-1:039638668084:loadbalancer/app/flask-express-backend-alb/e9501687a
aws_ecs_task_definition.frontend: Creating...
aws_lb_listener.backend: Creating...
aws_ecs_task_definition.frontend: Creation complete after 0s [id=flask-express-frontend]
aws_lb_listener.backend: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ap-south-1:039638668084:listener/app/flask-express-backend-alb/e950168
bb2387853f981751]
aws_ecs_service.backend: Creating...
aws_ecs_service.backend: Creation complete after 1s [id=arn:aws:ecs:ap-south-1:039638668084:service/flask-express-cluster/flask-express-backend]
aws_lb.frontend: Still creating... [02m40s elapsed]
aws_lb.frontend: Still creating... [02m50s elapsed]
aws_lb.frontend: Creation complete after 2m53s [id=arn:aws:elasticloadbalancing:ap-south-1:039638668084:loadbalancer/app/flask-express-frontend-alb/cf7697a
aws_lb_listener.frontend: Creating...
aws_lb_listener.frontend: Creation complete after 1s [id=arn:aws:elasticloadbalancing:ap-south-1:039638668084:listener/app/flask-express-frontend-alb/cf769
c/45cbb2902f8858fd]
aws_ecs_service.frontend: Creating...
aws_ecs_service.frontend: Creation complete after 2s [id=arn:aws:ecs:ap-south-1:039638668084:service/flask-express-cluster/flask-express-frontend]

Apply complete! Resources: 27 added, 0 changed, 0 destroyed.
```

```
C:\AWS\New folder\flask-express-ecs>cmd.exe /c "aws ecr get-login-passw
ord --region ap-south-1 | docker login --username AWS --password-stdin
039638668084.dkr.ecr.ap-south-1.amazonaws.com"
Login Succeeded
```

```
C:\AWS\New folder\flask-express-ecs>cd backend
```

```
C:\AWS\New folder\flask-express-ecs\backend>docker build -t flask-expre
ss-backend .
```

```
[+] Building 5.3s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 244B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.11-s  4.5s
=> [auth] library/python:pull token for registry-1.docker.io     0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:5be45db  0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:5be45db  0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 832B                                   0.0s
=> CACHED [2/5] WORKDIR /app                                     0.0s
```



```
C:\AWS\New folder\flask-express-ecs\backend>docker push 039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-express-backend:latest
The push refers to repository [039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-express-backend]
7b0cd198d40e: Already exists
f6ed45cd3980: Layer already exists
119d43eec815: Pushed
5b09819094bb: Layer already exists
50d27225ebbc: Layer already exists
1dc95a329252: Layer already exists
f79b4d6264a9: Layer already exists
3e731abb5c1d: Pushed
0b2bf04f68e9: Layer already exists
latest: digest: sha256:c49f3a13a7f90611008e93a7fdb862a4071932624ffadcc905a7588d87eee87 size: 856
```

```
C:\AWS\New folder\flask-express-ecs\frontend>docker build --build-arg BACKEND_URL=http://internal-flask-express-backend-alb-1699950942.ap-south-1.elb.amazonaws.com -t flask-express-frontend .
[+] Building 94.4s (10/10) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 167B 0.0s
=> [internal] load metadata for docker.io/library/node:18-alpin 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d66 0.2s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d66 0.2s
=> [internal] load build context 0.4s
```

```

C:\AWS\New folder\flask-express-ecs\frontend>docker tag flask-express-front
end:latest 039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-expr
ess-frontend:latest

C:\AWS\New folder\flask-express-ecs\frontend>docker push 039638668084.d
kr.ecr.ap-south-1.amazonaws.com/flask-express-frontend:latest
The push refers to repository [039638668084.dkr.ecr.ap-south-1.amazonaw
s.com/flask-express-frontend]
23b9152754e0: Pushed
1e5a4c89cee5: Pushed
ec429de88ed1: Pushed
25ff2da83641: Pushed
f18232174bc9: Pushed
0c71eaa06711: Pushed
d89721294a4e: Pushed
dd71dde834b5: Pushed
aacda7fb5e9f: Pushed
latest: digest: sha256:820b683e30c98f38501088ca11d8732efa34e501af21304e
7a34dda8086e1728 size: 856

```

```

PS C:\AWS\New folder\flask-express-ecs> aws ecs update-service --cluste
r $CLUSTER --service flask-express-frontend --force-new-deployment --re
gion ap-south-1
{
  "service": {
    "serviceArn": "arn:aws:ecs:ap-south-1:039638668084:service/flas
k-express-cluster/flask-express-frontend",
    "serviceName": "flask-express-frontend",
    "clusterArn": "arn:aws:ecs:ap-south-1:039638668084:cluster/flas
k-express-cluster",
    "loadBalancers": [
      {

```

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```

backend_alb_dns = "internal-flask-express-backend-alb-1699950942.ap-south-1.elb.amazonaws.com"
backend_ecr_url = "039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-express-backend"
cluster_name = "flask-express-cluster"
frontend_alb_dns = "flask-express-frontend-alb-761847007.ap-south-1.elb.amazonaws.com"
frontend_ecr_url = "039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-express-frontend"
frontend_url = "http://flask-express-frontend-alb-761847007.ap-south-1.elb.amazonaws.com"
PS C:\AWS\New folder\flask-express-ecs\terraform>

```

